

Wind LTSA Codebase Review

Why `createFit4` died on six data points, what changed to fix it, and which 110 of the 137 files in this folder are no longer part of the pipeline.

Fix applied & written to disk

Cleanup plan awaiting approval

6 files changed

1 file added

01 – THE FAILURE

The error was six steps downstream of the real problem

MATLAB reported a curve-fitting failure. The actual fault was that the wind file the code silently picked up did not overlap your LTSA period — and the code had no way to tell you that.

01 **getWindParams.m sets a folder, not a file**

`p.harp.WindFolder` points at the CCMP tree: `Z:\Wind_deltaTF\Wind_Data\SOCAL\`

02 **callLTSA.m51 tries five filename patterns in that folder**

The first hit wins. A leftover `SOCALBwindvec.mat` from an earlier CCMP run matched.

03 **Your explicit override was checked last, not first**

`p.windfile` sat in the sixth `elseif`, below all five auto-search patterns — so it could never win while any CCMP file existed.

04 **The "Pick Wind File" dialog never fired**

That prompt only appears when *nothing* matches. Something matched, so the run continued on the wrong wind.

05 **intersect() found almost no shared timestamps**

`iXm` came back with fewer than six entries. No warning: an empty intersection is a legal result.

06 **poly5 needs six points**

`createFit4` fits a fifth-order polynomial. With five or fewer points, `fit()` throws *"Insufficient data"* — the message you saw.

SECOND BUG FOUND ON THE WAY

`p.dLate = datenum([2026 0 0 0 0 0])` does not mean 2026. A zero month and zero day roll *backwards* — that expression evaluates to **30 November 2025**. Any LTSA data after that date was being silently discarded by `callTSA*51.m`. Rewritten as `datenum(2030,1,1)`.

02 — WHAT I CHANGED

One shared wind loader, and a guard that fails loudly

The wind-loading block was copy-pasted verbatim into all three band readers. It now lives in one place, honours your override first, and reports what it did.

FILE	CHANGE
<code>loadWindData.m</code> new	Shared wind loader. Checks <code>p.windfile</code> first , then the folder search, then prompts. Skips duplicate paths, sorts and de-duplicates timestamps, and prints the file path, sample count, date range, cadence and overlap with the LTSA period. Returns a <code>windInfo</code> provenance struct.
<code>callTSA51.m</code> <code>callTSA151.m</code> <code>callTSAm51.m</code>	~50 lines of triplicated wind code replaced with one call to <code>loadWindData(p.time)</code> . Stores <code>p.windInfo</code> alongside the saved workspace.
<code>getWindParams.m</code>	New <code>p.windSource</code> label and a documented <code>p.windfile</code> override block, with the NDBC 46053 paths pre-written and commented out. New <code>p.minFitPts</code> (default 20). <code>p.dLate</code> corrected.
<code>WindLTSA_v51.m</code>	New pre-fit check that prints wind source, file, series range, and matched-point count per band — then raises <code>WindLTSA:tooFewPoints</code> with a diagnostic checklist instead of dying inside <code>fit()</code> . Also stops the <code>p = Psave</code> restore from discarding wind provenance and any file you just picked.

To run against the NDBC buoy

Uncomment the `ndbcStation` block in `getWindParams.m` and set the years to match your deployment. Or just run it — the loader now prints the CCMP file it found and stops with a readable error, and if you clear `p.harp.WindFolder` you get the file picker back.

PLEASE VERIFY THIS ONE

The old code had a hard-coded special case: `strcmp([Proj,Site], 'SOCALB')` skipped the 6-hourly → hourly interpolation. The new loader measures the actual sample interval instead, which is what makes 10-minute NDBC data work at all. **If your SOCALB CCMP file is 6-hourly, it will now be interpolated where it previously was not** — more matched points, slightly different fits. The console prints the measured cadence, so you can confirm which case you're in before trusting a re-run against old results.

Twenty-seven files do all the work

WindLTSA_v51.m is a script, not a function — it opens with `clear variables` and communicates through `global p`. This is the whole reachable call graph, in execution order.

<code>getWindParams</code>	Build the global <code>p</code> struct — every path, band toggle and output filename	<code>reads</code> <code>HARpdataSummaryWIND.csv</code> (unused)
<code>getTFx</code> <code>getKTF</code>	Pick and parse the hydrophone transfer function	<code>getKTF</code> only when <code>p.usek = 'y'</code>
<code>callLTSA151</code> <code>callLTSAm51</code> <code>callLTSA51</code>	Read LTSA binaries → hourly mean spectra per band, apply TF	<code>read_ltsahead</code>
<code>loadWindData</code>	Load and align the wind time series new	—
<code>NoiseModelnew</code>	Knudsen-8 ocean noise model curves by sea state	<code>AMCatten</code>
<code>createFit4</code> <code>createFit5</code>	<code>poly5</code> robust fit, then 2σ outlier rejection twice over	<code>mintersect</code>
<code>WindSort51</code>	Bin the cleaned data into Beaufort wind-force bins	—
<code>WNRegress1</code>	Noise vs $\log(\text{wind})$ regression per frequency	<code>createFit2</code> , <code>TheilSen</code>
<code>tffmake51</code> <code>tffmake151</code> <code>tffmakeK</code>	Build the corrected transfer function and write the <code>.tf</code> files	<code>tffigmake</code> , <code>tffigmake1</code> , <code>tffigmakeKAIT</code>
<code>addsub51</code> <code>movebad</code>	Interactive add/remove of points; shunt rejected runs to BAD/	<code>selectdataA</code> , <code>selectdataS</code>

Most of the folder is per-site clones

The dominant pattern is copying the whole main script to change one line. v51, v51X, v51Y, v51Y (1), v100, v100x and v100y differ mainly in the hard-coded Proj/Site/Dep1 line — and v51Y vs v51Y (1) differ in *that line alone*.

GROUP	FILES	FINDING	SUGGESTED
WindLTSA mains	9	v50 → v100y are per-site clones of one script. v100 carries an ice-CSV block whose result, icedata, is never used again.	merge into one
callTSA readers	8	49 vs 51 differ by one hunk each; callTSA149 and callTSA151 are byte-identical apart from a comment.	keep 51 only
WindSort / addsub / tfmake / tfmakel	8	All four 49/51 pairs are byte-identical .	delete 49s
WindTimeSeries	14	v2/v2x/v2y/v2z and v3/v3x/v3y differ only in year range and site strings. Live ones are v4 (CCMP) and _NDBC.	parameterise
selectdata	4	Third-party file, copied four times to rename the function and change one brush-size constant.	keep 2
WhichProjSite	3	WhichProjSite.m is a superset; the (1) and (2) copies are browser-download duplicates missing site blocks.	delete copies
Knudsen / noise model	9	NoiseModelnew is live. UltimateKnudsen4–8 is the model's development history; OceanNoiseModel is the JASA figure script.	archive
Post-processing (TFCorrAVG×3, TFFinal...)	6	Hand-run downstream scripts. Not called by anything, but not dead — they consume the _TFCorr.mat files v51 writes.	keep, label
Scratch (Untitled2, TEMP, painting, Wind.m...)	~14	Wind.m still contains macOS Dropbox paths from a different machine.	delete
Unreferenced third-party (rdmseed, rgb, linspecer...)	5	rdmseed.m is a 979-line miniSEED reader with zero references anywhere.	delete
Graphics/	~100	Roughly 1.2 GB of figures. Single files up to 93 MB (VsFreq.eps).	move out of git
Github/, OLDVersions/, BatchCode/	50	Three parallel histories. Github/ is the JASA public release and has drifted; BatchCode/WindLTSA_v51.m is an <i>unrelated</i> file sharing the entry point's name.	separate repo

What will bite next

Ordered by how likely each is to produce a wrong answer rather than an error. The silent ones are the dangerous ones.

A bare `load()` overwrites `p` mid-run

SILENT

`WindLTSA_v51.m` line ~114 calls `load(...)` with no variable list, right after `callTSA51` returns. It dumps roughly forty names — including `p`, `Proj`, `Site`, `fs0` — into the script workspace. The other three loads name their variables; this one doesn't. High-band path only, so it hasn't bitten you yet.

Function names don't match filenames

CONFUSING

`WindSort51.m` declares function `WindSort49`. Same for `addsub51`, `tfmake51`, `tfmake151`, and all three `callTSA*51` files. MATLAB dispatches on filename so it runs, but every search for the real definition fails.

Name collision with `BatchCode/`

SILENT

`BatchCode/WindLTSA_v51.m` is a different 13 KB script that uses `global PARAMS` and hard-codes `H:\Wind_TF\`. If that folder ever lands on the MATLAB path, which `WindLTSA_v51` runs depends on path order.

Whole-workspace saves

BLOAT

`save(file)` with no variable list, in the `cal` functions and three times inside the `v51` edit loop. Every stray variable is written on every iteration, and reloading one of these files re-injects all of them.

Eight hard-coded paths, one `drivelet`

PORTABILITY

A run needs `Z:`, `E:` and `G:` mounted at once, and `drivelet` only controls three of the eight. `p.ltsa.LTSAFolder` and both Google Drive paths ignore it entirely.

Producer and consumer disagree on where wind lives

ROOT CAUSE

`getWindParams` expects `<drive>:\Wind_deltaTF\Wind_Data\<Proj>\<year>\`.
`WindTimeSeriesv4` writes to the repo's own `WindTimeSeries_Results`;
`WindTimeSeries_NDBC` writes under `NDBC_Buoy\raw\46053`. Nothing reconciles them.

A CSV read that nothing uses

DEAD COST

`getWindParams` runs `readtable` on `HARPdataSummaryWIND.csv` every launch. Its only consumer is `getTF.m` — which `v51` doesn't call, since it uses `getTFx`.

Four phases, ordered so nothing breaks

Each phase stands alone and can be approved separately. The order matters: later phases assume the safety net from Phase 1 is in place.

PHASE 1 **Make failures legible**

low risk · no behaviour change

Nothing about the science changes. This phase only makes the code say what it is doing.

- A `runLog.m` that stamps every output with who ran it, when, on what machine, from which git commit, with which parameters, reading which inputs and writing which outputs — as a sidecar `.txt` next to each `.mat` and `.tf`.
- Extend the `windInfo` provenance idea to the TF and LTSA inputs.
- Validate `p` once at startup: every path exists, every toggle is 'y' or 'n', output folders are writable. Fail with a list, not on the first miss.
- Replace the bare `load()` at line ~114 and give every `save()` an explicit variable list.

PHASE 2 **Collapse the clones**

medium risk · verify against a known deployment

Delete what is provably identical; parameterise what differs by one line.

- Delete the four byte-identical 49 helpers and the `WhichProjSite (1)/(2)` duplicates outright.
- Turn `WindLTSA_v51` into a function taking `(Proj, Site, Dep1)`, with a thin runner script holding the one line you currently edit. The eight sibling mains then collapse into it.
- Same treatment for the eleven `WindTimeSeries` variants.
- Rename the function lines to match their filenames.
- Move `OLDVersions/`, `BatchCode/` and `Graphics/` out of the working tree.

PHASE 3 **One config, one wind convention**

medium risk · touches every path

- Lift all eight hard-coded paths into a single per-machine config that isn't in git, so the repo runs on more than one computer.
- Agree one on-disk layout for wind products and make `WindTimeSeriesv4`, `WindTimeSeries_NDBC` and `loadWindData` all use it.
- Have the wind writers record station, coordinates, cadence and source in the `.mat`, so `loadWindData` can report them rather than infer them.
- Drop the unused `HARPdataSummaryWIND.csv` read, or connect it to something.

PHASE 4 **A regression test you can trust**

low risk · highest long-term value

- Pick one deployment, freeze its inputs and its current `_TFCorr.mat` output as a reference.
- A script that re-runs it non-interactively and diffs against the reference within tolerance.
- Run it after every phase above, so "did I change the science?" stops being a judgement call.

What I need from you before going further

A

Does the fix run?

Re-run `WindLTSA_v51.m` and send me the console output from the two new blocks. That tells us whether the NDBC file lines up and what cadence it measured.

B

Is the SOCALB interpolation change acceptable?

See the note in section 02. If your CCMP SOCALB file is 6-hourly, results will shift slightly. I can restore the old behaviour behind a flag if you'd rather compare like for like first.

C

Which phases do you want, and in what order?

Phase 1 is safe to start immediately. Phase 2 deletes files, so I'd want a commit and a green run first.

D

Is Github/ still a live public release?

It has drifted from the root and contains a `getWindParams.m` newer than anything the root pipeline uses. If it's the JASA archive it should be a separate repo; if it's meant to track root, it needs syncing.

E

Who else runs this?

It changes how much of Phase 3 is worth doing. If it's only ever you on this machine, the config work matters less than the provenance work.

Survey covered 137 root `.m` files plus `BatchCode`, `Github` and `OLDVersions`. ·

Read-only apart from the six files listed in section 02.